A Multi-Threaded Broken-Link Checker for the Marian University Website: An Operating-Systems–Informed Design and Evaluation

Jonas Schaefer

Fall 2025

Abstract

After a large Cascade CMS migration, the Marian University website contains more than 1,500 public pages. Manually verifying that every hyperlink still works is unrealistic and error-prone. In this project I designed and implemented a Python-based broken-link checker that crawls the Marian University sitemap, extracts internal links, and tests each target using HTTP requests. The system is explicitly shaped by concepts from Operating Systems: it uses a thread pool to model concurrent workers, configurable scheduling (FIFO and Priority) to determine crawl order, and a thread-safe Least Recently Used (LRU) cache to avoid redundant network requests. The tool produces structured CSV reports that list per-page violations as well as aggregate run statistics such as throughput, cache hit ratio, and resource usage.

This paper explains the motivation for the tool, describes the system design and implementation, and links each design decision to concrete OS principles such as scheduling, concurrency control, caching, and resource management. Finally, I discuss a small evaluation based on real runs on the Marian University sitemap and reflect on risks, limitations, and directions for future work.

1 Introduction

Broken links reduce the usability and perceived quality of a website. On a university website, they can also have practical consequences: students may not find important forms, prospective students might hit outdated application links, and staff members might unknowingly link to content that no longer exists. After a recent content migration to Cascade CMS, the Marian University website contains over 1,500 public pages spread across many sections such as Admissions, Academics, and Student Life. Checking all links by hand would be tedious and would have to be repeated frequently as content changes.

At the same time, I was taking a course on Operating Systems and wanted to apply ideas such as threads, scheduling, caching, and resource management to a concrete, real-world problem. This project brings these two motivations together. I built a broken-link checker that runs as a multi-threaded crawler. Each thread behaves like a lightweight process that fetches pages, extracts internal URLs, and checks them. A scheduler determines in which order URLs are processed, and an LRU cache simulates memory-management trade-offs by caching link-check results across pages.

The main goals of the project are:

- to build a practical tool that scans the Marian University website and reports broken links in a structured way,
- to design the tool around core OS concepts such as threads, scheduling, and caching, and
- to evaluate the impact of different scheduling and caching configurations on throughput and resource usage.

This work makes the following contributions:

1. It implements a fully functional, multi-threaded crawler that can scan the complete

Marian University sitemap (over 1,500 pages) and report both broken links and Cascade CMS login errors.

2. It translates core Operating Systems concepts—threads, scheduling, caching, and concurrency control—into concrete design decisions for a real web-maintenance tool.
3. It empirically compares different scheduler and cache configurations (FIFO vs. Priority, cache vs. no cache) with respect to throughput, resource usage, and time-to-first-useful-result.
4. It delivers a practical maintenance artifact: CSV-based reports that can be used directly by the Marian University MarCom team to prioritize and fix broken links.

This paper focuses on the parts that were fully implemented: the sitemap-based crawler, FIFO and Priority scheduling, violation detection (including Cascade CMS login links), and a thread-safe LRU cache. A PageRank-based graph analysis was originally proposed as an extension but was not implemented, so it is not part of this report.

2 Operating Systems Background

This section briefly reviews the OS concepts that guided the design of the broken-link checker. I focus on threads and concurrency, scheduling, caching and LRU policies, and basic resource-management ideas such as rate limiting.

2.1 Threads and Concurrency

In a traditional time-sharing operating system, multiple processes or threads make progress "at the same time" on one CPU through context switching. Arpaci-Dusseau and Arpaci-Dusseau describe threads as a way for a single process to have multiple independent execution contexts that share the same address space [1]. Threads can increase responsiveness and throughput, but they introduce shared-state challenges such as race conditions and deadlocks.

For a web crawler, concurrency is especially attractive because the workload is I/O-bound: most of the time is spent waiting for HTTP responses. By using several threads, the program can overlap waiting periods and increase the number of URLs checked per second without needing multiple cores. In this project, I used Python's `ThreadPoolExecutor` to manage a fixed-size pool of worker threads. Each worker repeatedly takes the next URL from a shared queue, fetches the page, and records the results. Alternative designs, such as multiprocessing or fully asynchronous I/O, were considered but not adopted. Multiprocessing would incur higher overhead for inter-process communication and duplicate memory usage, which is unnecessary for a workload that is dominated by network latency rather than CPU time. Asynchronous I/O with an event loop could in principle offer similar benefits, but it would require a substantially different architecture and more complex coordination patterns. Given the moderate size of the Marian sitemap, a thread-pool model strikes a pragmatic balance between implementation complexity and performance.

2.2 Scheduling Policies

Scheduling decides which ready task gets the CPU next. In OS theory, there are many policies such as First-In-First-Out (FIFO), Shortest Job First, Priority Scheduling, and Round Robin [2, 3]. The choice of scheduler affects throughput, fairness, and potentially starvation.

In this project, the "tasks" are web pages to be crawled. I implemented two scheduling strategies:

- **FIFO**: URLs are processed in the order in which they appear in the sitemap.
- **Priority**: URLs are ordered by a simple heuristic based on path depth and URL length. Shorter and shallower paths (for example, `/admissions`) are crawled earlier than deep, parameterized URLs.

Instead of letting the OS scheduler decide arbitrarily which threads fetch which pages, the program explicitly orders URLs before they enter the frontier queue. In this way, the

scheduling policy is part of the application logic and can be compared across runs.

2.3 Caching and LRU Policies

Caching is a classic OS technique: keeping recently used data in a fast storage layer to avoid expensive accesses to slower storage. In memory hierarchies, LRU (Least Recently Used) is a common replacement policy that evicts the entry that has not been used for the longest time [1]. Caches are also widely used in middleware and web systems to avoid repeated work [2].

In a broken-link checker, the same target URL may appear on many different pages, especially for commonly used navigation links or footers. Without a cache, each occurrence would trigger a separate HTTP request, even if the target is identical. This not only wastes time but also puts unnecessary load on the server.

To address this, I implemented a thread-safe LRU cache that stores the result of `check_link` calls keyed by the normalized target URL. If a link to the same target appears again on another page, the program retrieves the cached result instead of performing another network request. The cache tracks its number of accesses, hits, and misses so that hit ratio can be included in the evaluation. Within the broader landscape of replacement policies, LRU was chosen because the crawler repeatedly encounters the same navigation and footer links within short time windows, which creates strong temporal locality. A FIFO policy would ignore recency and might evict still-useful entries, while a Least Frequently Used (LFU) policy would react more slowly to shifts in which sections of the site are being crawled. LRU therefore aligns well with the access pattern induced by sitemap-ordered and priority-ordered crawls.

2.4 Resource Management and Rate Limiting

Running a crawler against a production website involves ethical and technical risks. Opening too many concurrent connections or sending requests too quickly may degrade the site, especially if it is not designed for high-volume crawling. Both OS textbooks and middleware

literature emphasize the need to avoid over-committing resources and to respect external systems [2, 1].

In this project, resource management appears in two ways:

- the crawler uses a bounded thread pool with a configurable number of worker threads, and
- each thread calls a rate-limiting helper function that enforces a minimum delay between requests from that thread.

This design ensures that the total request rate stays within a reasonable range and that the system is polite to the Marian servers.

3 Problem Setting and Requirements

The target of the crawler is the public Marian University website accessible under `https://www.marian.edu`. The starting point for the crawl is the automatically generated XML sitemap at `/sitemap.xml`. This sitemap lists more than 1,500 URLs that represent the pages to be scanned.

From the perspective of the Marketing and Communications (MarCom) team, the key problem is to identify pages that contain broken links. Broken links include URLs that return HTTP status codes 404 (Not Found), 500 (Server Error), or any other code ≥ 400. In addition, there are specific Cascade CMS login URLs that should not appear on public content pages, such as direct links into the CMS backend.

Before coding, I specified the following functional requirements:

- **Sitemap-based crawl:** Read all URLs from the Marian sitemap and treat each one as a candidate page to visit.
- **Internal link extraction:** On each HTML page, extract all internal links (limited to allowed domains) from the main content area.

- **Broken-link detection:** For each internal link, perform an HTTP request with redirects allowed and classify targets with status ≥ 400 or network errors as broken.
- **Cascade login detection:** Mark links whose URL matches a configured pattern for Cascade CMS login endpoints as violations, even if they might technically return 200.
- **Reporting:** Produce a CSV summary of all crawled pages including violation counts, and a separate CSV with one row per violating link, so that MarCom can filter and fix problems.

In addition, I formulated non-functional requirements that reflect the OS focus:

- **Performance:** The system should be able to process at least 100 URLs within one minute on a standard development machine.
- **Politeness:** The crawler must respect server load limits using rate limiting and a reasonable number of threads.
- **Configurability:** Scheduling policy, cache mode, and other parameters should be configurable without changing code.
- **Reproducibility:** Each run should produce machine-readable logs and summaries that can be compared across experiments.

These requirements guided the design of the system and the shape of the configuration file.

4 System Design and Architecture

The overall architecture of the broken-link checker is modular. The main components are:

1. a sitemap parser that discovers the set of pages to crawl,
2. a scheduler that orders URLs according to the chosen policy,
3. a multi-threaded worker pool that fetches pages and extracts links,

4. a link checker that classifies targets as OK, broken, or Cascade login, and
5. a reporting layer that writes CSV summaries and a run summary.

The configuration is stored in a `config.yaml` file. It specifies the sitemap URL, a domain allowlist, crawler settings such as thread count and delay, checker options, and cache options. For example, the cache section allows switching between `"none"` and `"lru"` modes and sets the maximum cache size.

4.1 Sitemap Parsing

The sitemap parser reads the Marian sitemap XML and extracts all `<loc>` elements, which contain absolute URLs. It first tries to use `BeautifulSoup` with an XML parser, and falls back to the standard library's `ElementTree` if necessary. To improve robustness and avoid duplicates, each URL is normalized:

- `http://` is converted to `https://`,
- fragment identifiers (after #) are removed, and
- trailing slashes are stripped.

The result is written to `data/urls_initial.csv` with one URL per line. This CSV serves as the frontier for the crawler.

4.2 Scheduling and Frontier Management

Before the crawl starts, the program reads `urls_initial.csv` and passes the list of URLs to a scheduler function. The scheduler can operate in two modes: FIFO and Priority.

A helper function computes a priority score for each URL based on its path depth (number of `/` segments) and whether it has a query string. Shorter, shallower URLs without query parameters receive a lower score and thus higher priority. In priority mode, the list of URLs is sorted by this score and then fed into a thread-safe queue that acts as the frontier.

The frontier queue is consumed by worker threads. A separate visited set, protected by a lock, keeps track of which URLs have already been processed. This prevents duplicate work even if the sitemap or scheduling configuration accidentally contains duplicates.

4.3 Worker Threads and Rate Limiting

The core of the crawler is a function that runs in each worker thread. Each worker repeatedly:

1. retrieves a URL from the frontier queue,
2. checks and updates the visited set under a lock,
3. waits if necessary according to the rate limit,
4. sends an HTTP GET request using a per-thread session,
5. parses the HTML and extracts internal links, and
6. records results in shared lists and counters.

Rate limiting is implemented using a thread-local variable that stores the timestamp of the last request from that thread. Before each request, the thread computes how much time has elapsed since the last one and sleeps if the configured delay has not passed yet. This design keeps the delay per thread, which in practice spreads the load across time without needing a global lock.

Each request records the start and end timestamps in UTC, the HTTP status code, the final URL after redirects, the content type, and the elapsed time in milliseconds. Only responses with a content type that includes `text/html` are parsed for links; other content types are skipped but still logged.

4.4 Link Extraction

To avoid being distracted by repetitive navigation elements such as headers and footers, the link extractor focuses on the main content area of each page. Using `BeautifulSoup`, it first removes typical layout elements like `<header>`, `<nav>`, `<footer>`, and elements with classes like `site-header` and `site-footer`. Then it tries to find a `<main>` element or a `<div>` with id or class suggesting content. If none is found, it falls back to scanning the whole document.

From this main area, the function iterates over all anchor tags with an `href` attribute. For each link, it resolves relative URLs against the current page URL, filters to only internal domains based on the allowlist, and normalizes the result for counting. Special care is taken for `mailto:` links: internal Marian email addresses are treated as internal "links" so that pages can be analyzed for contact consistency, while external email domains are ignored.

The function returns two values: a list of normalized internal links and a count of internal links. The count can either include duplicates or count only unique targets, depending on configuration. In the current setup, duplicates are counted to give a sense of how often certain internal links appear.

4.5 Violation Detection and Cascade Logins

Violation detection happens on two levels: per-link and per-page.

On the per-link level, the crawler must decide whether a given internal URL is acceptable, broken, or a Cascade login. For Cascade logins, the configuration contains a list of patterns (for example, a CMS hostname or a specific login path) that are considered invalid on public pages. If any of these patterns appears as a substring in the link URL, the link is classified as a Cascade login violation without making a network request.

Otherwise, the program calls `check_link`, which performs an HTTP GET request with `allow_redirects = True`. If the final status code is 400 or higher, the link is classified

as a broken link. If the response is a redirect, the behavior depends on configuration: in the Marian setup, redirects are treated as OK as long as the final status code is below 400. Network-related `RequestException`s are also treated as broken links and include a short note indicating the error.

On the per-page level, the worker aggregates all violations encountered while checking links from that page. It records a summary string such as `"none"`, `"broken_link"`, `"cascade_login"`, or `"broken_link+cascade_login"`, and a count of violations for that page. These two fields are added to the main crawl CSV so that pages with many problems can be identified quickly.

4.6 Thread-Safe LRU Cache

To reduce redundant link checks, the project includes a custom LRU cache implemented in Python using `OrderedDict` and a `Lock`. The cache is generic, storing values of type `V` keyed by type `K`, but in this context the keys are normalized link targets and the values are the tuples returned by `check_link`.

The cache publicly exposes two methods:

- `get(key)`: increments the access counter, returns the value if present, and updates the recency ordering; otherwise increments the miss counter and returns `None`.
- `set(key, value)`: inserts or updates the key and evicts the least recently used entry if the maximum size would be exceeded.

All operations that mutate internal state are wrapped in a lock to make the cache safe for concurrent access from multiple threads. The cache also computes simple statistics: total accesses, hits, misses, and hit ratio. These values are read at the end of the crawl and written into the run summary.

An important design choice is the normalization of cache keys. To avoid creating separate entries for the same logical target, query strings and fragments are stripped from URLs, and

scheme and host are converted to lowercase. This means that URLs like `/apply?utm=123` and `/apply?utm=456` share a cache entry, which is desirable for link integrity.

4.7 Reporting and Run Summary

At the end of a crawl, the program produces three main CSV outputs:

- **links_multithread.csv**: one row per page, with columns for URL, status, time in milliseconds, thread name, start and end timestamps, error message if any, number of internal links found, final URL after redirects, content type, page-level violation summary, and violation count.
- **violations_links.csv**: one row per violating link, with columns for source page URL, link URL, violation type (`"broken_link"` or `"cascade_login"`), status code (if applicable), final URL, and a short note.
- **run_summary.csv**: one row per crawl run, appended across runs. Columns include the timestamp, scheduler mode, number of threads, delay, total URLs, total duration, URLs per second, total broken links, total Cascade logins, number of pages with violations, total links found, cache mode, cache size, cache accesses, hits, misses, hit ratio, and optional CPU and memory statistics.

The run summary is especially important for evaluation, because it allows comparing different configurations such as FIFO versus Priority scheduling or cache versus no cache modes.

5 Implementation Details

The implementation is written in Python 3.12 and organized into several modules inside a `src/` directory. The entry point is `main.py`, which reads the configuration, triggers sitemap parsing, and then calls the crawler for real runs.

5.1 Configuration File

The `config.yaml` file centralizes options for the crawler. Relevant keys include:

- `sitemap_url`: the starting sitemap URL, here `"https://www.marian.edu/sitemap.xml"`.
- `domain_allowlist`: a list of domain suffixes considered internal, for example `"marian.edu"` and `"marian.cascadecms.com"`.
- `threads`, `timeout`, `delay`: core crawler parameters.
- `scheduler`: either `"fifo"` or `"priority"`.
- `extract_links`, `count_duplicates`: flags for link extraction.
- `checker`: a nested section with `cascade_login_patterns`, `only_internal`, `treat_redirect_as_ok`, and `max_links_per_page`.
- `cache`: a nested section with `mode` (`"none"` or `"lru"`) and `max_size`.
- `output_dir`, `data_dir`, and `csv_delimiter`: output-related settings.
- `mock_mode`: a flag that enables loading a local test HTML file instead of crawling the real website, used in early development.

This file allows switching between experimental setups without touching the code, which makes evaluation easier and less error-prone.

5.2 Main Driver

The `main` function first loads the YAML configuration and runs the sitemap parser. If `mock_mode` is enabled, it loads a single HTML file from disk, extracts links, and writes them to a sample CSV. This mode was useful during Week 1 to verify that parsing and link extraction worked correctly without contacting the real servers.

When `mock_mode` is disabled, the program reads the list of URLs from `urls_initial.csv`,

optionally truncates it if `max_urls` is set, and calls `crawl_all` with the selected URLs and configuration. All later behavior—scheduling, concurrency, checking, caching, and reporting—is encapsulated inside the crawler.

6 Evaluation and Results

The evaluation of the system focuses on three aspects:

1. whether the crawler can successfully scan the full Marian sitemap and produce meaningful reports,
2. how Priority scheduling compares to FIFO in practice, and
3. how enabling the LRU cache affects performance and resource usage.

Due to limited time and the desire to avoid stressing the production servers, I did not run an extensive factorial experiment. Instead, I used several representative runs over the full sitemap (around 1,600 URLs) and inspected the resulting reports and summary CSV.

Table 1 summarizes representative quantitative results for different scheduler and cache configurations.

Configuration	URLs/s	Duration (s)	Cache Hit Ratio	Broken Links
FIFO + no cache	3.1	523.8	–	358
Priority + no cache	3.2	520.0	–	359
FIFO + LRU cache	8.9	176.0	0.66	358
Priority + LRU cache	9.2	175.8	0.66	358

Table 1: Representative performance metrics for different scheduler and cache configurations. The numerical values are based on full-sitemap runs with 12 threads and a delay of 0.5 seconds.

6.1 Correctness and Coverage

In real-mode runs, the sitemap parser produced a URL list with more than 1,500 entries. The crawler was able to process this entire list without errors, as indicated by the total

URL count in the run summary and by the coverage of `links_multithread.csv`. The `violations_links.csv` file contained hundreds of rows, including both broken links (status ≥ 400 or network errors) and Cascade login URLs that were still present on some pages.

An important correctness property is that the cache does not change the semantics of the results. To verify this, I compared runs with `cache.mode = "none"` and `cache.mode = "lru"` on the same set of URLs. While the request timings and thread interleavings differ, the resulting violation counts and types per page match when sorted by URL. This confirms that the cache only reduces redundant requests but does not hide or introduce broken links.

6.2 Priority Scheduling in Practice

The Priority scheduler is a simple heuristic: it uses path depth, query parameters, and URL length as a proxy for importance. Top-level pages such as `/`, `/admissions`, or `/academics` are crawled earlier than deep content pages or URLs with long query strings.

In effect, this implies that the most visible sections of the site appear earlier in the logs and in the violation reports.

For the MarCom team, this is useful because they can begin reviewing broken links on high-level pages even before a full crawl has finished.

Quantitatively, the Priority scheduler did not significantly reduce the total wall-clock runtime compared to FIFO, because both policies ultimately process the same set of URLs under the same thread and delay constraints. Its contribution lies instead in reordering which parts of the site complete first. This behavior mirrors priority-based CPU schedulers in operating systems, where response time for important tasks can be improved without necessarily changing total throughput.

6.3 Cache Performance

The LRU cache is most effective when there are many repeated internal links, which is typical for a CMS-driven site with shared navigation and footer components. In one representative run over roughly 1,600 URLs with Priority scheduling and 12 threads, the run summary recorded around 6,900 cache accesses, of which approximately 4,500 were hits and 2,300 misses. This corresponds to a hit ratio of about 0.66.

This means that roughly two thirds of all link checks were served from the cache without making a network request. Because HTTP requests dominate the cost of the workload, this reduction in network traffic has a measurable effect on throughput. In that same run, the crawler processed on the order of 9 URLs per second, finishing the full sitemap in under three minutes. In contrast, longer runs under heavier system load or without cache showed significantly lower throughput values, sometimes around 3 URLs per second.

The cache also has an effect on resource usage. While it consumes additional memory to store cached results, the LRU eviction policy and a configurable maximum size keep this usage bounded. At the same time, CPU usage stays moderate because threads mainly wait on I/O rather than performing heavy computation. The optional integration with `psutil` records average CPU percentage and resident memory usage in megabytes, which confirms that the crawler stays within acceptable limits on a typical development machine.

The analysis itself is constrained by methodological limitations. To avoid placing excessive load on production servers, only a small number of full-sitemap runs were executed, and they were performed within a narrow time window. Longer-term measurements across different days or weeks, as well as stress tests under intentionally higher concurrency, were deliberately avoided. As a result, the reported throughput and hit-ratio figures should be interpreted as indicative rather than statistically robust.

7 Risks, Limitations, and Ethics

Like any system that interacts with external services, this broken-link checker has risks and limitations. During the design, I tried to identify them and either mitigate them or make them explicit.

7.1 Caching-Related Risks

A first concern is the risk of stale data: could the cache cause the crawler to report outdated link states? In this implementation, the cache is only valid within a single run. Once the program exits, the cache is discarded, and a new run starts fresh. Because a typical Marian crawl completes in minutes, it is unlikely that link statuses change during one run, so the risk of intra-run staleness is low.

Another concern is the risk of incorrect cache values. If the program were to cache incorrect results due to a bug in `check_link` or key normalization, it could hide broken links. To reduce this risk, I tested the cache by comparing runs with cache enabled and disabled and ensuring that the resulting violations matched. Key normalization is applied consistently in both the cache and the link-extraction logic.

Memory usage is also a risk: a cache without limits could use unbounded memory and degrade the system. To prevent this, the LRU cache has a configurable maximum size. When the number of stored entries exceeds this limit, the least recently used entry is evicted. This guarantees that memory usage remains bounded.

Finally, concurrency can introduce subtle race conditions. The cache uses a single lock to protect all operations that modify internal state. While this slightly reduces potential parallelism, it greatly simplifies reasoning about correctness and avoids the risk of inconsistent statistics or corrupted entries.

7.2 Crawling Ethics and Server Load

From an ethical perspective, crawling a production website should always be done carefully. Sending too many simultaneous requests or ignoring `robots.txt` rules can unintentionally perform a denial-of-service attack. For Marian, I followed these principles:

- limit the number of worker threads to a moderate value (such as 12),
- enforce a per-thread delay between requests,
- restrict the crawler strictly to the allowed domains and sitemap URLs, and
- coordinate crawls with the MarCom and IT teams when needed.

The crawler does not attempt to discover new URLs beyond the sitemap, and it does not try to access authentication-protected or internal-only pages. It is designed to help maintain the public site, not to test security or probe for hidden content.

7.3 Limitations

There are several limitations to the current implementation:

- **No JavaScript evaluation:** Links generated dynamically by JavaScript are not seen by the crawler, because it only parses static HTML.
- **Focus on link integrity only:** The tool does not check images, PDFs, or other assets, nor does it validate content structure or accessibility.
- **No async I/O:** While asynchronous I/O could theoretically increase throughput for highly I/O-bound workloads, integrating `asyncio` would require a substantial rewrite of the crawler. Given the moderate size of the Marian site and the performance achieved with threads and caching, I chose not to implement this at this time.
- **Simple scheduling heuristic:** The Priority scheduler uses a very simple rule based on URL structure. It does not integrate analytics or access logs and therefore only

approximates “importance”.

These limitations point towards possible future extensions rather than fundamental flaws for the current use case.

8 Future Work

From both an academic and an institutional perspective, not all potential extensions are equally urgent. Some directions would provide high impact for Marian University with relatively modest implementation effort, while others would require substantial re-engineering of the system. There are several natural directions for extending the broken-link checker beyond its current state. At a high level, they can be grouped by expected impact and implementation effort.

One obvious extension is to add support for additional resource types, such as images, PDFs, and other downloadable files. This would require extending the link-extraction logic to consider `<img>` and `<a>` tags pointing to non-HTML content and deciding on appropriate rules for what counts as a broken resource.

Another direction is to integrate asynchronous I/O. A reimplementation of the worker pipeline using `asyncio` and `aiohttp` could increase throughput by avoiding thread scheduling overhead and making better use of the event loop. However, this would also introduce new complexity, such as managing async-safe data structures and mixing synchronous and asynchronous components.

On the scheduling side, one could experiment with additional policies or dynamic prioritization based on observed error rates. For example, if certain sections of the site historically contain more broken links, the scheduler could prioritize them in subsequent runs. Round-robin scheduling across different subsections might also be interesting for fairness, although for a single sitemap it is less relevant than in multi-user OS contexts.

Finally, the reporting could be extended with a small web-based dashboard where non-technical users can filter, sort, and mark violations as fixed, connecting the crawler more directly to MarCom's daily workflow.

9 Concluding Observations

This project demonstrates how core Operating Systems concepts can guide the design of a practical tool for web maintenance. Starting from the concrete problem of detecting broken links on the Marian University website, I built a sitemap-based crawler that uses a thread pool, configurable scheduling, and a thread-safe LRU cache to efficiently scan more than around 1,600 URLs pages.

Design choices such as FIFO versus Priority scheduling, a bounded thread pool, rate limiting, and cache modes are not arbitrary implementation details but explicit applications of OS principles from threads, scheduling, and caching theory. The resulting system produces CSV reports that are immediately useful to the MarCom team and can be analyzed quantitatively using the run summary.

While there is still room for extensions—such as async I/O, richer scheduling policies, and more resource types—the current implementation already provides both institutional value and academic insight. It shows that ideas from Operating Systems are not just abstract concepts but powerful tools for structuring and evaluating real-world software.

References

[1] Remzi H. Arpaci-Dusseau and Andrea C. Arpaci-Dusseau. *Operating Systems: Three Easy Pieces*. Arpaci-Dusseau Books, 2018.

[2] Max Hailperin. *Operating Systems and Middleware: Supporting Controlled Interaction*. Revised Edition 1.1.6, Gustavus Adolphus College, 2014.

[3] Robert Sedgewick and Kevin Wayne. *Algorithms*, Fourth Edition. Addison-Wesley, 2011.

[4] Abraham Silberschatz, Peter B. Galvin, and Greg Gagne. *Operating System Concepts*, Ninth Edition. Wiley, 2013.

[5] Marian University. Marian University official website. `https://www.marian.edu`.